Kelas 11-A

Lapak Buah 11-A

Paket siswa: panduan pengerjaan bertahap dan lembar kerja untuk membuat website lapak buah selama satu minggu

Mata pelajaran	Pemrograman Web dan Pemrograman Berbasis Teks, Grafis dan Multimedia
Waktu	Senin 19 sampai Kamis 22 Oktober 2026
Bagian 1	Panduan pengerjaan proyek (Pertemuan 1 sampai 5)
Bagian 2	Lembar kerja: tebakan, jawaban, uji silang, refleksi
Cara pakai	Kerjakan sambil membuka folder starter. Isi lembar kerja dengan kata-katamu sendiri.



Nama: _____ No. absen: _____

Panduan Pengerjaan Proyek "Lapak Buah 11-A"

Panduan ini membimbing kamu membuat website lapak buah selama satu minggu. Panduannya sengaja **tidak memberikan kode jadi**. Kamu akan diberi arah, contoh dari kasus lain, dan cara memeriksa hasil. Kodenya kamu tulis sendiri, supaya kamu benar-benar paham dan bisa menjelaskannya.

Baca ini dulu

Apa yang kamu buat

Website lapak buah dengan fitur:

- Katalog buah yang dibuat dari data
- Tombol "Tambah ke nota", nota yang bisa diubah jumlahnya
- Diskon otomatis, pembulatan, total, dan kembalian
- Kupon diskon (Level 2)
- Tampilan rapi di laptop dan HP, plus mode gelap

Semua memakai materi yang sudah kamu pelajari: variabel, operator, konversi tipe, function, array, object, Map, DOM, dan ES Module.

Lima aturan main

1. **Tebak dulu, baru jalankan.** Tulis tebakanmu di lembar kerja sebelum menjalankan kode. Salah tebak itu bagus, di situlah kamu belajar.
2. **Ketik sendiri.** Jangan menyalin kode teman atau hasil AI. Contoh di panduan ini memakai kasus lain (siswa, absen, ongkir) supaya kamu menerjemahkannya sendiri ke proyek.
3. **Satu langkah, satu pengujian.** Selesai satu fungsi, langsung jalankan `node uji-logika.mjs` atau lihat di browser. Jangan menumpuk lima fungsi lalu baru mencoba.
4. **Tiga tanya dulu.** Baca pesan error, tanya teman sebangku, baru tanya guru. Guru akan membalas dengan pertanyaan balik.
5. **Simpan dan isi exit ticket** di akhir setiap pertemuan.

Cara membaca panduan

Setiap langkah punya bagian yang sama:

- **Tujuan**: apa yang selesai setelah langkah ini
- **Konsep**: materi yang dipakai, lengkap dengan file asalnya di folder `PWEB-11A`
- **Tebak dulu**: pertanyaan sebelum mencoba (jawab di lembar kerja)
- **Kerjakan**: apa yang harus kamu tulis
- **Cek hasil**: tanda bahwa langkahmu benar
- **Pertanyaan**: jawab di lembar kerja, dengan kata-katamu sendiri
- **Jika macet**: bantuan bertahap, mulai dari petunjuk kecil

Peta file proyek

```
starter/
├── index.html          struktur halaman (SUDAH ADA, kamu baca dan pahami)
├── css/style.css       tampilan (sebagian kamu tulis)
├── uji-logika.mjs      pemeriksa otomatis (tidak diubah)
└── js/
    ├── data.js         DATA produk                          kamu tulis
    ├── helper.js       fungsi hitung-hitungan dan format    kamu tulis
    ├── keranjang.js    isi nota (Map)                       kamu tulis
    ├── riwayat.js      catatan pesanan (bonus)              kamu tulis
    └── main.js         DOM dan event                        kamu tulis
```

Alur data:



`main.js` adalah satu-satunya file yang menyentuh halaman (DOM). File lain hanya berisi data dan logika, sehingga bisa diperiksa lewat terminal tanpa browser.

Alat bantu

Alat	Cara	Gunanya
Live Server	Klik kanan `index.html`, Open with Live Server	Membuka halaman lewat alamat `http://...` (wajib untuk ES Module)
Muat ulang paksa	`Ctrl + Shift + R` (Mac: `Cmd + Shift + R`)	Mengabaikan cache jika perubahan tidak terlihat
DevTools	`F12`, tab **Console** dan **Elements**	Membaca error, memeriksa elemen dan CSS
Terminal VS Code	Menu Terminal, New Terminal	Menjalankan `node uji-logika.mjs`
`console.log`	Tambahkan di kodemu	Mengintip nilai variabel. Hapus jika sudah selesai

Cara membaca pesan error

Pesan di Console	Artinya	Periksa
`blocked by CORS policy ... origin 'null'`	Halaman dibuka dengan klik dua kali (`file://`)	Buka lewat Live Server
`does not provide an export named 'xxx'`	Nama yang di-import tidak ada di file asalnya	Ejaan nama, kata `export` di file asal
`Cannot read properties of null (reading 'addEventListener')`	`querySelector` tidak menemukan elemennya	Ejaan id (`#tambah-1`), dan apakah elemennya sudah dibuat (urutan render)
`xxx is not a function`	Memanggil sesuatu yang bukan function	Ejaan, kurung, apakah sudah di-import
`xxx is not defined`	Variabel belum dibuat atau belum di-import	Ejaan, `import` di bagian atas file
`undefined` tampil di halaman	Function lupa `return`, atau variabel belum diisi	Baris `return`

Pesan di Console	Artinya	Periksa
`NaN` tampil di halaman	Operasi angka dengan nilai yang bukan angka	Apakah ada string yang ikut dihitung (`Number(...)`)

Pertemuan 1: Kerangka halaman dan data

Senin 19 Oktober 2026, 13:05 sampai 15:00. Target: halaman statis rapi dan `data.js` berisi 6 produk.

Langkah 1.1 Siapkan dan jalankan

Tujuan. Halaman starter terbuka lewat alamat `http://...` tanpa error merah di Console.

Konsep. ES Module (`modularisasi/`, `ESModule/`). `<script type="module">` memungkinkan `import` dan `export` di browser.

Tebak dulu. Jika kamu klik dua kali `index.html` (alamatnya `file:///...`), halaman berjalan normal atau tidak? Tulis tebakanmu.

Kerjakan.

1. Salin folder `starter` ke laptopmu, ganti namanya menjadi `lapak-buah-NAMAMU`.
2. Buka folder itu di VS Code (File, Open Folder).
3. Klik kanan `index.html`, pilih **Open with Live Server**.
4. Lihat alamat di browser. Harus diawali `http://127.0.0.1:5500` atau `http://localhost`.
5. Tekan `F12`, buka tab **Console**.
6. Buka terminal VS Code, ketik `node -v` lalu `node uji-logika.mjs`. Banyak tanda ❌ itu **normal** karena kamu belum menulis apa-apa.

Cek hasil. Halaman terbuka (tampilannya masih polos), Console tidak merah, tes menampilkan ringkasan WAJIB / LEVEL 2 / BONUS.

Pertanyaan. Jelaskan dengan kata-katamu: kenapa halaman dengan `<script type="module">` harus dibuka lewat server?

Jika macet.

- Tidak ada Live Server: di terminal ketik `python3 -m http.server 5500` (Windows: `py -m http.server 5500`), lalu buka `http://localhost:5500`.
- `node` tidak dikenali: Node.js belum terpasang, bilang ke guru.

Langkah 1.2 Baca HTML dan buat Peta ID

Tujuan. Kamu tahu elemen mana yang akan dipegang JavaScript.

Konsep. `id` dipakai JavaScript untuk menemukan satu elemen (`querySelector("#nama")`). `class` dipakai CSS untuk menata banyak elemen. Tag semantik (`header`, `main`, `section`, `aside`, `footer`) memberi arti pada bagian halaman. Lihat `dom/index.html`.

Tebak dulu. Buka `js/main.js`, lihat bagian "MEMEGANG ELEMEN". Dari nama variabelnya, tebak 2 elemen yang isinya akan diganti teks dan 2 elemen yang akan diberi event klik.

Kerjakan. Buka `index.html`, temukan elemen yang punya `id`. Isi tabel **Peta ID** di lembar kerja (minimal 8 baris): id, jenis elemen, dan perkiraan apa yang JavaScript lakukan padanya.

Cek hasil. Tabelmu terisi dan tebakanmu sudah dicocokkan dengan `main.js`.

Pertanyaan. Apa yang terjadi di `main.js` kalau id `#tombol-bayar` di HTML kamu ganti menjadi `#tombol-bayar2`?

Langkah 1.3 Header (CSS)

Tujuan. Bilah hijau tua di atas halaman: logo di kiri, dua tombol di kanan, menempel saat halaman di-scroll.

Konsep. Variabel CSS (`--nama`, dipakai `var(--nama)`) dan flexbox. Mini-lesson M2 dari guru.

Contoh dari kasus lain (kartu nilai siswa, bukan header):

```
.kartu-nilai {
  display: flex;                  /* anak-anaknya berjajar dalam satu baris */
  justify-content: space-between; /* sebar: satu ke kiri, satu ke kanan */
  align-items: center;            /* sejajarkan di tengah secara vertikal */
  gap: 12px;                      /* jarak antar anak */
  padding: 12px 16px;
  background: var(--permukaan);   /* nilai dari variabel di :root */
}
```

Tebak dulu. Menurutmu `justify-content` mengatur arah mana, mendatar atau menurun? Dan `align-items`?

Kerjakan. Di `css/style.css`, isi empat blok `LANGKAH 1.3`: `.header`, `.header-isi`, `.logo`, `.header-aksi`. Petunjuk properti ada di komentar tiap blok.

Cek hasil. Header tampil seperti target. Scroll halaman: header tetap di atas.

Pertanyaan. Apa bedanya `justify-content` dan `align-items`? Apa yang terjadi bila `position: sticky` dihapus?

Jika macet.

1. Klik kanan elemen di browser, pilih **Inspect**. Di panel **Styles** kamu bisa mencentang atau mematikan properti untuk melihat efeknya.
2. Pastikan nama class di CSS sama persis dengan di HTML (`.header-isi`, bukan `.headerIsi`).
3. Cek tanda `;` di setiap akhir properti dan `}` di akhir blok.

Langkah 1.4 Papan pembuka dan tata letak dua kolom

Tujuan. Bagian pembuka (judul dan stiker) tertata. Halaman terbagi dua kolom: katalog di kiri, nota di kanan.

Konsep. CSS Grid.

Contoh dari kasus lain:

```
.dua-kolom {
  display: grid;
  grid-template-columns: 1fr 200px; /* kolom kiri fleksibel, kolom kanan 200px */
  gap: 16px;
  align-items: start;               /* tiap kolom setinggi isinya sendiri */
}
```

Tebak dulu. Pada contoh di atas, jika jendela browser dilebarkan, kolom mana yang ikut melebar?

Kerjakan. Isi blok `LANGKAH 1.4`: `.papan`, `.papan-teks p`, `.papan-info`, `.stiker`, `.tata-letak`, `.alat`, dan `select, input`. Stiker "Diskon 5%" yang ada di HTML adalah contoh agar kamu bisa melihat hasilnya.

Cek hasil. Dua kolom terlihat. Nota (kertas krem) ada di kolom kanan. Kecilkan jendela: bagian kirinya menyempit, bukan mendorong nota keluar layar.

Pertanyaan. Komentar di `.tata-letak` meminta `minmax(0, 1fr)`, bukan `1fr` saja. Cari tahu apa bedanya, lalu tulis jawabanmu.

Langkah 1.5 Katalog dan kartu produk

Tujuan. Kartu produk rapi: foto berlatar warna, label harga menempel di foto, nama, info, dan tombol.

Konsep. `repeat(auto-fill, minmax(...))`, `position: relative` dan `absolute`, variabel CSS yang diisi dari HTML (`style="--warna: ..."`).

Contoh dari kasus lain:

```
.galeri {
  display: grid;
  /* "sebanyak mungkin kolom selebar minimal 150px, sisanya dibagi rata" */
  grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
  gap: 12px;
}
```

Tebak dulu. Jika `.galeri` berada di layar selebar 600 px, berapa kolom yang muncul? Kalau layar 300 px?

Kerjakan. Isi blok yang masih `TODO` di bagian `LANGKAH 1.5`: `.katalog`, `.kartu`, `.kartu-foto`, `.kartu-foto img`, `.kartu-isi`, `.kartu-info`, `.kartu-isi .tombol`, `.kartu.habis .kartu-foto img`. Label harga dan stempel "Habis" sudah disediakan guru, baca saja.

Cek hasil.

- Kartu apel tampil dengan latar merah muda, label harga di sudut kiri bawah foto.
- Ubah `class="kartu"` di HTML menjadi `class="kartu habis"`: gambar menjadi abu-abu dan muncul stempel "Habis". Kembalikan setelah dicoba.
- Lebarkan dan kecilkan jendela. Jumlah kolom berubah sendiri.

Pertanyaan. Kenapa `.kartu-foto` harus `position: relative`? (Petunjuk: lihat `.harga-tag`.) Dari mana warna latar foto diambil?

Langkah 1.6 Data produk

Tujuan. `js/data.js` berisi minimal 6 produk valid.

Konsep. Array berisi object, named export. Lihat `struktur-data/array.js`, `struktur-data/object.js`, dan `ESModule/config.mjs` (array `mahasiswa`).

Contoh dari kasus lain (`ESModule/config.mjs`):

```
export const mahasiswa = [
  { nama: "Maulana", nilai: 90 },
  { nama: "Dimas", nilai: 80 },
];
```

Tebak dulu. Jika harga ditulis `"18.000"` (string, pakai titik), apa hasil `"18.000" * 2`? Coba di Console browser.

Kerjakan. Lengkapi `data.js` mengikuti aturan di komentarnya. Pastikan ada minimal satu produk dengan `stok: 0`.

Agar hasil hitunganmu di Pertemuan 3 dan 4 sama dengan contoh di panduan, pakai data berikut. Kamu boleh mengubahnya setelah semuanya selesai.

id	name	price	category	stok	gambar	warna
1	Apel	32000	Impor	12	`apel-merah.svg` (gambarAlt: `apel-hijau.svg`)	`#ffe1de`
2	Pisang Ambon	18000	Lokal	20	`pisang.svg`	`#fff1b8`
3	Mangga Harum Manis	25000	Lokal	4	`mangga.svg`	`#ffe7bd`
4	Nanas Madu	15000	Lokal	0	`nanas.svg`	`#f4f0c4`
5	Jeruk Medan	22000	Lokal	15	`jeruk.svg`	`#ffddc2`
6	Anggur Ungu	60000	Impor	6	`anggur.svg`	`#e9ddff`

Cek hasil. Di terminal jalankan `node uji-logika.mjs`. Bagian **DATA** harus ✅ semua (4 tes).

Pertanyaan. Kenapa `id` harus unik? Di bagian mana `id` produk akan dipakai nanti? (Petunjuk: lihat komentar `buatKartu` di `main.js`.)

Exit ticket Pertemuan 1: kerjakan di lembar kerja, lalu simpan pekerjaanmu.

Pertemuan 2: Katalog dinamis

Selasa 20 Oktober 2026, 09:40 sampai 10:20. Target: kartu buah tampil dari `data.js`.

Langkah 2.1 `formatRupiah`

Tujuan. `formatRupiah(32000)` menghasilkan `"Rp32.000"`, dan `formatRupiah()` menghasilkan `"Rp0"`.

Konsep. Function expression dan parameter default: `function-experession.js`, `function.js`. Operator `+` sebagai penggabung string: `pertemuan-4.pdf`.

Contoh dari kasus lain:

```
const formatPersen = function (angka = 0) {
  return angka + "%";
};
formatPersen(15); // "15%"
formatPersen();   // "0%"
```

Tebak dulu. Ketik `(32000).toLocaleString("id-ID")` di Console browser. Apa hasilnya dan apa tipe datanya (`typeof`)?

Kerjakan. Tulis `formatRupiah` di `js/helper.js` sebagai **function expression** dengan parameter default.

Cek hasil. `node uji-logika.mjs`: tes `formatRupiah` ✅.

Pertanyaan. Apa bedanya function expression dan function declaration saat dipanggil sebelum barisnya? (Petunjuk: `function-experession.js` baris atas.)

Langkah 2.2 `buatKartu`

Tujuan. `buatKartu(item)` menerima satu object produk dan mengembalikan string HTML satu kartu.

Konsep. Object destructuring (`object.js`), template literal, ternary. Mini-lesson M3.

Contoh dari kasus lain:

```
const buatBarisSiswa = (siswa) => {
  const { nama, nilai } = siswa;
  const lulus = nilai >= 75;
  return `<li class="${lulus ? "lulus" : "remedial"}">${nama}: ${nilai}</li>`;
};
buatBarisSiswa({ nama: "Dimas", nilai: 80 });
// '<li class="lulus">Dimas: 80</li>'
```

Tebak dulu. Apa tipe data hasil `buatBarisSiswa(...)`: object, array, atau string?

Kerjakan. Di `js/main.js`, isi `buatKartu`. Baca daftar "yang harus berubah mengikuti data" di komentarnya. Struktur HTML-nya sama dengan kartu contoh di `index.html`.

Cek hasil. Sementara tambahkan `console.log(buatKartu(produk[0]))` di bawah fungsi itu, simpan, dan lihat hasilnya di Console. HTML-nya harus mirip kartu contoh. Hapus `console.log` sesudahnya.

Pertanyaan. Kenapa `buatKartu` mengembalikan string dan tidak langsung mengisi halaman? (Petunjuk: bisa dipakai untuk setiap produk, dan mudah diuji.)

Langkah 2.3 `renderKatalog` dan `mulai`

Tujuan. Semua produk dari `data.js` tampil sebagai kartu.

Konsep. `forEach`, `innerHTML`, pola "render, lalu pasang event". Mini-lesson M3 dan M4.

Contoh dari kasus lain:

```
const daftarSiswa = [
  { nama: "Rizky", nilai: 80 },
  { nama: "Dimas", nilai: 90 },
];
const daftar = document.querySelector("#daftar");

let html = "";                        // wadah teks, mulai dari kosong
daftarSiswa.forEach((siswa) => {
  html += buatBarisSiswa(siswa);      // tambahkan satu baris tiap putaran
});
daftar.innerHTML = html;              // baru ditulis ke halaman, satu kali
```

Tebak dulu. Jika baris `let html = "";` diganti `let html;` (tanpa nilai), apa isi `html` di akhir? (Coba, hasilnya mengejutkan.)

Kerjakan. Isi `renderKatalog` untuk **Langkah A sampai C** saja. Langkah D (pasang event tombol) dikerjakan di Pertemuan 3. Lalu isi `mulai()` agar memanggil `renderKatalog()` dan `renderNota()`.

Cek hasil.

- Kartu contoh statis hilang, diganti 6 kartu dari `data.js`.
- Ubah `stok` sebuah produk menjadi `0`, muat ulang: kartunya otomatis menjadi "Habis".
- Teks "6 jenis buah ditampilkan" muncul.

Pertanyaan. Kenapa event tidak bisa dipasang pada tombol di dalam kartu sebelum `katalog.innerHTML = html`? Apa pesan error yang akan muncul?

Exit ticket Pertemuan 2: kerjakan di lembar kerja.

Pertemuan 3: Nota belanja dengan Map

Selasa 20 Oktober 2026, 12:30 sampai 13:40. Target: tombol "Tambah ke nota" berfungsi.

Langkah 3.1 `keranjang.js`

Tujuan. Enam fungsi nota lulus semua tes KERANJANG.

Konsep. Map: `set`, `get`, `has`, `delete`, `clear`, `forEach`. Key boleh berupa object. Lihat `struktur-data/map.js` dan `struktur-data/real-case-map.js`. Mini-lesson M5.

Contoh dari kasus lain (mencatat kehadiran):

```
const absen = new Map();
const rizky = { nama: "Rizky" };

absen.set(rizky, 1);                          // key = object, value = jumlah hadir
absen.has(rizky);                             // true
absen.get(rizky);                             // 1
absen.set(rizky, absen.get(rizky) + 1);       // naik jadi 2
absen.delete(rizky);                          // dibuang
absen.forEach((jumlah, siswa) => { /* ... */ }); // urutan parameter: (value, key)
```

Tebak dulu.

```
const a = { id: 1 };
const b = { id: 1 };
const peta = new Map();
peta.set(a, "x");
console.log(peta.has(a), peta.has(b), peta.size);
```

Tulis tiga hasilnya sebelum menjalankan di Console.

Kerjakan (berpasangan). Kerjakan satu fungsi, jalankan `node uji-logika.mjs`, lihat ✅, baru lanjut. Pengetik dan pengamat bertukar tiap 2 fungsi. Urutan: `tambahItem`, `kurangiItem`, `hapusItem`, `kosongkanKeranjang`, `hitungSubtotal`, `hitungJumlahItem`.

Cek hasil. Bagian **KERANJANG** di tes ✅ semua (9 tes).

Pertanyaan.

1. Kenapa `peta.has(b)` bernilai `false` padahal isi `a` dan `b` sama?
2. Apa urutan parameter callback `Map.forEach`? Apa bedanya dengan `Array.forEach`?
3. Apa yang terjadi bila `tambahItem` tidak mengecek stok?

Jika macet.

1. Tulis dulu dengan kalimat biasa: "Kalau item belum ada di nota, jumlah sekarang 0. Kalau sudah ada, ambil jumlahnya." Lalu terjemahkan ke `has` dan `get`.
2. Baca pesan ❌ di terminal. Pesannya menyebut nilai yang diharapkan dan nilai yang kamu hasilkan.
3. Pasang `console.log(keranjang)` di dalam fungsimu untuk melihat isi Map.

Langkah 3.2 `tambahKeNota` dan daftar item di nota

Tujuan. `renderNota` menampilkan daftar item dari Map.

Konsep. Alur data satu arah:

```
klik tombol
   │
   ▼
tambahKeNota(item) ──► tambahItem(item)      (keranjang.js, ubah data)
   │
   ▼
renderNota() ──► keranjang.forEach(...) ──► string HTML ──► daftarNota.innerHTML
   │
   └──► pasang event untuk tombol −, +, Hapus yang baru dibuat
```

Setiap kali data berubah, HTML dibangun ulang dari data. Kamu tidak mengutak-atik elemen satu per satu.

Tebak dulu. Pada baris nota, apa yang harus unik untuk setiap produk supaya `querySelector` bisa menemukan tombol yang benar? Dari mana nilainya diambil?

Kerjakan. Di `main.js`, isi `tambahKeNota`, lalu bagian **Langkah 3.2** di `renderNota` (poin a sampai e). Satu baris nota memakai elemen berikut (nama class sudah ada di CSS):

Elemen	Isi
`img`	Foto kecil produk
`div.baris-info`	`strong` nama produk dan `span` "Rp... x N kg"
`div.stepper`	`button` kurangi, `output` jumlah, `button` tambah
`span.baris-harga`	Harga x jumlah
`button.hapus`	Teks "Hapus"

Beri tombol id yang unik: `kurang-${item.id}`, `tambah-nota-${item.id}`, `hapus-${item.id}`. Tombol yang hanya berisi simbol (`−`, `+`) perlu `aria-label` agar terbaca oleh pembaca layar.

Cek hasil. Pada tahap ini tombol di katalog belum berfungsi. Uji sementara: di `main.js`, tepat di atas baris `mulai();` tulis `tambahItem(produk[0]);`. Pastikan `mulai()` memanggil `renderNota()`. Nota harus menampilkan Apel 1 kg. Hapus baris uji itu sesudahnya.

Pertanyaan. Kenapa parameter `keranjang.forEach` ditulis `(jumlah, item)` dan bukan `(item, jumlah)`?

Langkah 3.3 Pasang event tombol katalog dan uji manual

Tujuan. Tombol "Tambah ke nota" di setiap kartu berfungsi.

Kerjakan. Kembali ke `renderKatalog`, kerjakan **Langkah D** (pasang event klik tiap tombol). Jalankan skenario berikut:

No	Skenario	Hasil yang benar
1	Klik "Tambah ke nota" pada Apel 3 kali	Nota: Apel 3 kg. Angka di tombol Nota menjadi 3
2	Klik tombol `−` pada Apel	Jadi 2 kg

No	Skenario	Hasil yang benar
3	Klik "Tambah ke nota" pada Mangga (stok 4) sebanyak 5 kali	Jumlah berhenti di 4 kg dan muncul pesan stok tinggal 4 kg. Tombol `+` di nota ikut nonaktif
4	Klik `−` pada item yang tinggal 1 kg	Item hilang dari nota
5	Klik "Hapus" pada item dengan 3 kg	Item hilang sekaligus
6	Cek kartu produk dengan stok 0	Tombol nonaktif, tidak bisa diklik

Exit ticket Pertemuan 3: kerjakan di lembar kerja.

Pertemuan 4: Diskon, pembayaran, dan kupon

Rabu 21 Oktober 2026, 12:30 sampai 13:40. Target: total dan kembalian benar.

Langkah 4.1 Diskon bertingkat

Tujuan. Tiga fungsi lulus tes: `formatPotongan`, `hitungPersenDiskon`, `hitungDiskon`.

Konsep. Arrow function, ternary bersusun, operator perbandingan. Lihat bonus bertingkat di `challange.js` (gaji >= 5.000.000 dapat 10%).

Contoh dari kasus lain (ongkir bertingkat):

```
const hitungOngkir = (jarakKm) => (jarakKm >= 10 ? 15000 : jarakKm >= 5 ? 8000 : 0);
hitungOngkir(12); // 15000
hitungOngkir(5);  // 8000
hitungOngkir(2);  // 0
```

Tebak dulu (sebelum mengetik kode). Hitung dengan tangan dan isi tabel di lembar kerja:

Subtotal	Persen diskon	Diskon (Rp)
60.000	?	?
74.999	?	?
75.000	?	?
150.000	?	?

Kerjakan. Di `helper.js` isi `formatPotongan`, `hitungPersenDiskon`, dan `hitungDiskon`. Gunakan konstanta `BATAS_DISKON_SEDANG` dan `BATAS_DISKON_TINGGI`, jangan menulis angka 75000 dan 150000 langsung.

Cek hasil. Tes `formatPotongan`, `hitungPersenDiskon`, `hitungDiskon` ✅, dan hasilnya sama dengan tabel tebakanmu.

Pertanyaan. Rumus diskon di proyek ini ditulis `subtotal * persen / 100` (kalikan dulu, baru bagi). Coba `3 * 5 / 100` dan `3 * (5 / 100)` di Console. Apa bedanya? Kenapa urutan pertama lebih aman untuk uang?

Langkah 4.2 `hitungRingkasan`

Tujuan. Satu fungsi yang mengembalikan seluruh angka nota dalam bentuk object.

Konsep. Return object, parameter default berupa function, operator modulus (`%`). Lihat `pertemuan-4.pdf` (modulus) dan `function.js` (parameter default).

Tebak dulu.

1. Berapa `73150 % 100`?
2. Apa hasil `((a, f = () => 0) => f(a))(5)`? Tulis tebakanmu, lalu cek di Console.

Hitung dengan tangan dulu. Lengkapi tabel ini di lembar kerja:

Subtotal	Kupon	Diskon	Potongan kupon	Sebelum pembulatan	Pembulatan	Total
100.000	tidak ada	?	?	?	?	?
77.000	tidak ada	?	?	?	?	?
80.000	10% (Level 2)	?	?	?	?	?

Kerjakan. Tulis `hitungRingkasan(subtotal, fungsiKupon)` sesuai spesifikasi di komentarnya. Parameter kedua harus punya nilai default berupa function yang selalu mengembalikan 0.

Cek hasil. Tes `hitungRingkasan` ✅ dan sama dengan tabelmu. (Tes yang memakai kupon baru lulus setelah Langkah 4.5.)

Pertanyaan. Untuk apa pembulatan ke bawah kelipatan Rp100 di toko sungguhan? Kenapa memakai `%` dan bukan pembagian biasa?

Langkah 4.3 Ringkasan di nota dan kembalian

Tujuan. Nota menampilkan subtotal, diskon, kupon, pembulatan, total. Kolom bayar menghitung kembalian.

Konsep. Object destructuring dari hasil function, konversi eksplisit: `konversi-implisit.js`, `konversi-eksplisit.js`. Mini-lesson M6.

Tebak dulu. Kolom bayar berisi `100000`. Tulis hasil:

```
inputBayar.value + 1000      // ?
Number(inputBayar.value) + 1000  // ?
inputBayar.value - 1000      // ?
Number("")                   // ?
```

Kerjakan.

1. Di `renderNota`, kerjakan poin f sampai i (ringkasan harga).
2. Isi `renderKembalian`.
3. Pasang event `input` pada `inputBayar`. Perhatikan: `renderKembalian` dikirim **tanpa tanda kurung**.

Cek hasil.

- Tambahkan Apel 3 kg dan Anggur 1 kg (subtotal Rp156.000): diskon 10%, total Rp140.400.
- Isi kolom bayar `100000`: tertulis "Uang kurang Rp40.400" berwarna merah. Isi `150000`: "Kembalian Rp9.600" berwarna hijau.
- Kosongkan kolom bayar: pesan hilang.

Pertanyaan.

1. Kenapa `inputBayar.value` selalu string walaupun `type="number"`?
2. Apa bedanya `addEventListener("input", renderKembalian)` dengan `addEventListener("input", renderKembalian())`?

Langkah 4.4 Tombol Bayar

Tujuan. Klik Bayar memeriksa uang, mengurangi stok, dan mengosongkan nota.

Kerjakan. Isi event `tombolBayar` mengikuti sembilan langkah di komentarnya. Pada langkah 6, baris `riwayat.catatPesanan(pesanan)` boleh dikosongkan dulu karena bonus, tetapi `nomorPesanan++` tetap dikerjakan.

```
klik Bayar
   ├─ nota kosong?        ──► toast, berhenti
   ├─ uang kurang?        ──► toast, berhenti
   └─ cukup ──► buat object pesanan ──► kurangi stok ──► reset nota ──► render ulang ──► toast
```

Cek hasil.

Skenario	Hasil yang benar
Klik Bayar saat nota kosong	Pesan nota masih kosong
Bayar kurang dari total	Pesan uang kurang, nota tetap
Bayar pas atau lebih	Pesan pesanan dibayar dan kembalian. Nota kosong. Stok di kartu berkurang
Beli seluruh stok Mangga lalu bayar	Kartu Mangga berubah menjadi "Habis"

Pertanyaan. Kenapa stok dikurangi **setelah** pembayaran, bukan saat item dimasukkan ke nota?

Langkah 4.5 Kupon (Level 2)

Tujuan. Kode kupon memotong harga. Tes **LEVEL 2** lulus semua (5 tes).

Konsep. Function yang mengembalikan function (`first-class-citizen.js`, bagian `multiplier`), Map yang berisi function (`struktur-data/map.js` dan `array-object-function.js`). Mini-lesson M7.

Contoh dari kasus lain (pajak):

```
const buatPajak = (persen) => (harga) => (harga * persen) / 100;

const ppn11 = buatPajak(11);   // ppn11 sekarang adalah function
ppn11(100000);                 // 11000

const daftarPajak = new Map([["PPN11", buatPajak(11)], ["PPN5", buatPajak(5)]]);
daftarPajak.get("PPN11")(100000); // 11000  (ambil function dari Map, langsung panggil)
```

Tebak dulu. Pada `buatPajak(11)`, di mana angka `11` "disimpan" sehingga masih diingat saat `ppn11(100000)` dipanggil jauh sesudahnya?

Kerjakan.

1. Tulis `buatKupon(persen)` di `helper.js`.
2. Isi `daftarKupon` dengan tiga kupon: `HEMAT5` (5%), `HEMAT10` (10%), `KELAS11A` (15%).
3. Di `main.js`, isi event `tombolKupon`.

Cek hasil. Tes LEVEL 2 ✅. Di halaman: kupon `kelas11a` (huruf kecil) tetap diterima, kupon `ngawur` menampilkan pesan merah, nota ikut berubah.

Pertanyaan.

1. Kenapa `daftarKupon.get("HEMAT10")(200000)` bisa langsung dipanggil seperti function?
2. Jika ingin menambah kupon `BUAH20` (20%), file mana yang diubah dan berapa baris?

Exit ticket Pertemuan 4: kerjakan di lembar kerja.

Pertemuan 5: Finishing, uji silang, wawancara kode

Kamis 22 Oktober 2026, 10:20 sampai 11:40.

Langkah 5.1 Blok A (semua siswa)

Tujuan. Halaman responsif, mode gelap, sapaan, dan papan info berfungsi.

Konsep. Media query (`@media`), `className` dan `textContent` (`dom/event.html`), function declaration (`function.js`, `return-value.js`).

Kerjakan.

1. **CSS responsif.** Isi tiga bagian `LANGKAH 5.1` di akhir `style.css`. Uji dengan DevTools: tekan `Ctrl + Shift + M` (mode perangkat), pilih lebar 375 px dan 1366 px.
2. **Mode gelap.** Isi event `tombolTema` di `main.js`. CSS gelap sudah disediakan (`body.gelap`).
3. **Sapaan.** Tulis `sapaByWaktu(jam)` di `helper.js` sebagai function declaration, lalu pakai di `mulai()`.
4. **Papan info.** Isi `renderPapanInfo`. Angka persen dan batas rupiah diambil dari konstanta dan fungsi di `helper.js`, bukan ditulis manual.

Cek hasil.

Cek	Hasil yang benar
Lebar 375 px	Tidak ada scroll mendatar. Katalog 2 kolom. Nota di bawah katalog
Lebar 1366 px	Dua kolom besar. Nota di kanan
Klik "Mode gelap"	Warna berubah, teks tombol menjadi "Mode terang". Klik lagi kembali terang
Sapaan	Sesuai jam laptop (pagi, siang, sore, malam)
Ubah `BATAS_DISKON_SEDANG` jadi `100000`	Papan info ikut berubah tanpa mengubah `main.js`

Pertanyaan. Kenapa mode gelap cukup menimpa nilai variabel CSS dan tidak perlu menulis ulang semua aturan?

Langkah 5.2 Blok B: pilih fitur lanjutan

Pilih sesuai kemampuanmu. Minimal satu bila Blok A dan Langkah 4.5 sudah selesai.

B1. Filter dan urutan (Mini-lesson M8)

Konsep. `sort` dan `filter` menerima function. `Array.from` untuk menyalin array. `array-method.js`.

Tebak dulu.

```
const angka = [10, 9, 100, 1];
angka.sort();                    // ?
angka.sort((a, b) => a - b);     // ?
```

Kerjakan. Di `renderKatalog` (bagian BONUS), tambahkan filter kategori dan urutan harga. Pasang event `change` untuk `pilihKategori` dan `pilihUrutan`.

Cek hasil. Kategori "Impor" hanya menampilkan 2 buah (jika datamu sama dengan contoh). Urutan "Harga termurah" benar. Array `produk` asli tidak berubah urutannya.

Pertanyaan. Kenapa harus `Array.from(produk)` dulu sebelum `sort`?

B2. Riwayat pesanan

Konsep. `push`, `shift`, `pop` (`manipulasi-array.js`), `Array.from` dan `reverse` (`array.js`, `array-method.js`), array destructuring di parameter.

Kerjakan. Tulis tiga fungsi di `riwayat.js`, isi `renderRiwayat`, event `tombolBatal`, dan aktifkan langkah riwayat di event Bayar.

Cek hasil. Tes **BONUS** ✅ (5 tes). Riwayat maksimal 5 pesanan, yang paling baru di atas. Batalkan pesanan terakhir mengembalikan stok.

Pertanyaan. `reverse()` mengubah array aslinya. Bagaimana kamu menampilkan urutan terbaru di atas tanpa merusak urutan asli?

B3. Ganti warna apel

Konsep. Array destructuring untuk menukar dua nilai: `[a, b] = [b, a]`. Ubah `img.src` (`dom/event.html`).

Kerjakan. Pada kartu yang punya `gambarAlt`, pasang event pada tombol "Ganti warna" untuk menukar `gambar` dan `gambarAlt`, lalu mengganti `src` gambar.

Langkah 5.3 Uji silang

Tujuan. Menemukan bug di karya temanmu tanpa memperbaikinya.

Kerjakan. Tukar laptop dengan teman. Jalankan 12 skenario di lembar kerja (bagian "Uji silang"). Catat setiap kegagalan: skenario nomor berapa, apa yang terjadi, apa yang seharusnya. Kembalikan laptop dan perbaiki bug dari temanmu pada karyamu sendiri.

Langkah 5.4 Wawancara kode dan refleksi

Tujuan. Menunjukkan bahwa kamu memahami kodemu sendiri.

Persiapan. Pilih tiga fungsi yang kamu tulis sendiri. Untuk masing-masing, latih menjelaskan dengan pola:

> "Fungsi ini menerima ... , melakukan ... , mengembalikan ... . Dipakai di ... ."

Saat wawancara, guru akan:

1. Menunjuk satu fungsi dan memintamu menjelaskan.
2. Mengajukan satu pertanyaan konsep dari materi.
3. Meminta satu **perubahan langsung** (sekitar 2 menit), misalnya:
   - ganti batas diskon 5% menjadi Rp100.000
   - tambahkan kupon `BUAH20` (20%)

- batasi pembelian maksimal 5 kg per jenis
- ubah warna utama tombol menjadi biru

Siswa yang menulis sendiri kodenya tahu persis file mana yang diubah.

Refleksi (tulis di lembar kerja, 3 sampai 5 kalimat): bagian apa yang paling kamu pahami, bagian apa yang masih membingungkan, dan apa rencanamu mempelajarinya.

Lampiran A: Contekan CSS yang dipakai

Kebutuhan	Properti
Menyusun anak dalam satu baris atau kolom	`display: flex`, `flex-direction`, `justify-content`, `align-items`, `gap`, `flex-wrap`
Menyusun dalam kolom dan baris	`display: grid`, `grid-template-columns`, `gap`
Kolom otomatis menyesuaikan layar	`repeat(auto-fill, minmax(210px, 1fr))`
Jarak dalam dan luar	`padding`, `margin`, `margin-inline: auto` (tengahkan)
Garis tepi dan sudut	`border`, `border-radius`, `overflow: hidden`
Warna dari variabel	`color`, `background`, `var(--nama)`
Posisi	`position: relative`, `absolute`, `sticky`, `top`, `left`, `bottom`
Gambar	`width`, `max-width: 100%`, `filter: grayscale(1)`, `opacity`
Aturan khusus layar	`@media (max-width: 900px) { ... }`
Keadaan elemen	`:hover`, `:disabled`, `:focus-visible`

Lampiran B: Materi asal di repositori

Topik	Buka berkas
Operator, ternary	`operator.js`, `pertemuan-4.pdf`
Konversi tipe	`konversi-implisit.js`, `konversi-eksplisit.js`
Function	`function.js`, `return-value.js`, `function-experession.js`, `arrow-function.js`
Function sebagai nilai	`first-class-citizen.js`, `struktur-data/array-object-function.js`
Array	`struktur-data/array.js`, `manipulasi-array.js`, `array-method.js`, `array-destructuring.js`
Object	`struktur-data/object.js`
Map	`struktur-data/map.js`, `real-case-map.js`
DOM dan event	`dom/index.html`, `dom/event.html`
Modul	`modularisasi/`, `ESModule/`
Latihan serupa diskon	`challange.js`, `arrow-function.js` (hitungNilai)

Lampiran C: Istilah

Istilah	Arti singkat
Render	Membuat tampilan (HTML) dari data
State	Data yang berubah selama halaman terbuka (kategori aktif, kupon aktif)
Callback	Function yang dikirim ke function lain untuk dijalankan nanti (`forEach`, `addEventListener`, `setTimeout`, `sort`)
Closure	Function yang masih "mengingat" variabel dari tempat ia dibuat (`buatKupon`)
Module	File JavaScript yang memakai `import` dan `export`
Destructuring	Mengambil isi object atau array ke variabel terpisah
Pseudo-element	Elemen buatan CSS (`::before`, `::after`)
Media query	Aturan CSS yang hanya berlaku pada kondisi layar tertentu

Lembar Kerja Siswa: Proyek "Lapak Buah 11-A"

Nama: _____ No. absen: _____ Kelas: 11-A

Isi lembar ini sambil mengerjakan `02-panduan-siswa.md`. Tulis jawaban dengan kata-katamu sendiri. **Bagian "Tebak" diisi SEBELUM menjalankan kode.**

Pertemuan 1: Senin 19 Oktober 2026

Tebakan pembuka

Dari demo guru, fitur mana yang menurutmu memakai materi apa?

Fitur yang kamu lihat	Materi yang dipakai (tebakanmu)
1.	
2.	
3.	

Langkah 1.1 sampai 1.2

Tebak: jika `index.html` dibuka dengan klik dua kali, apa yang terjadi?

Jawab: kenapa halaman dengan `<script type="module">` harus dibuka lewat server?

Peta ID (minimal 8 baris):

id	Jenis elemen	JavaScript akan melakukan apa padanya?
`#sapaan`		

id	Jenis elemen	JavaScript akan melakukan apa padanya?

Jawab: apa yang terjadi di `main.js` jika id `#tombol-bayar` di HTML diganti `#tombol-bayar2`?

Langkah 1.3 sampai 1.5

Tebak (1.3): `justify-content` mengatur arah mendatar atau menurun? `align-items`?

Jawab (1.4): apa beda `minmax(0, 1fr)` dan `1fr`?

Tebak (1.5): jumlah kolom `repeat(auto-fill, minmax(150px, 1fr))` (jarak 12 px) di layar 600 px: _____ kolom. Di layar 300 px: _____ kolom.

Jawab (1.5): kenapa `.kartu-foto` harus `position: relative`? Dari mana warna latar fotonya diambil?

Langkah 1.6

Tebak: hasil `"18.000" * 2` adalah _____ . (Cek di Console. Sesuai tebakanmu?)

Jawab: kenapa `id` setiap produk harus unik?

Checkpoint Pertemuan 1 (diisi guru)

- ☐ Halaman terbuka lewat `http://` dan Console tanpa error
- ☐ Header, papan, dan dua kolom tampil
- ☐ Kartu contoh rapi dan state "habis" bisa dicoba
- ☐ `node uji-logika.mjs`: bagian DATA 4 dari 4 ✅

Paraf guru: _____

Exit ticket Pertemuan 1

1. Kenapa proyek ini harus dibuka lewat Live Server, bukan klik dua kali?

---

2. Tulis satu object produk dengan key `name`, `price`, `category`, `stok`.

---

3. Sebutkan dua keuntungan variabel CSS (`--aksi`) dibanding menulis warna langsung.

Pertemuan 2: Selasa 20 Oktober 2026 (Jam ke-3)

Tebak dan jawab

Tebak (2.1): `(32000).toLocaleString("id-ID")` menghasilkan _____ dengan tipe data _____ .

Jawab (2.1): apa beda function expression dan function declaration saat dipanggil sebelum barisnya?

Tebak (2.2): tipe data hasil `buatBarisSiswa(...)` (contoh di panduan): _____

Jawab (2.2): kenapa `buatKartu` mengembalikan string dan tidak langsung mengisi halaman?

Tebak (2.3): jika `let html;` (tanpa `= ""`), isi `html` di akhir diawali teks _____

Jawab (2.3): apa yang terjadi bila event dipasang sebelum `katalog.innerHTML = html`? Tulis pesan errornya.

Checkpoint Pertemuan 2 (diisi guru)

- [ ] `formatRupiah` lulus tes
- [ ] Kartu tampil dari `data.js` (bukan dari HTML)
- [ ] Produk `stok: 0` otomatis "Habis" setelah halaman dimuat ulang

Paraf guru: _____

Exit ticket Pertemuan 2

1. `buatKartu(item)` mengembalikan tipe data apa? _____
2. Kenapa tombol di kartu tidak bisa diberi `addEventListener` sebelum `innerHTML` diisi?

---

3. `formatRupiah()` tanpa argumen menghasilkan _____ . Fitur apa yang membuatnya begitu? _____

Pertemuan 3: Selasa 20 Oktober 2026 (Jam ke-6 dan 7)

Tebak dan jawab

Tebak (3.1):

```
const a = { id: 1 };
const b = { id: 1 };
const peta = new Map();
peta.set(a, "x");
console.log(peta.has(a), peta.has(b), peta.size);
```

Hasil: _____ , _____ , _____

Jawab: kenapa `peta.has(b)` bernilai `false`?

Jawab: apa urutan parameter callback `Map.forEach`? Apa bedanya dengan `Array.forEach`?

Jawab: apa yang terjadi bila `tambahItem` tidak mengecek stok?

Jawab (3.2): kenapa parameter `keranjang.forEach` ditulis `(jumlah, item)`?

Uji manual nota (Langkah 3.3)

No	Skenario	Berhasil?
1	Tambah Apel 3 kali, jadi 3 kg, badge 3	
2	Tombol `−` pada Apel, jadi 2 kg	
3	Tambah Mangga (stok 4) 5 kali, berhenti di 4 kg dan ada pesan	
4	`−` pada item 1 kg, item hilang	
5	"Hapus" pada item 3 kg, hilang sekaligus	
6	Produk stok 0, tombol nonaktif	

Checkpoint Pertemuan 3 (diisi guru)

- ☐ `node uji-logika.mjs`: bagian KERANJANG 9 dari 9 ✅
- ☐ Tombol "Tambah ke nota", `−`, `+`, "Hapus" berfungsi
- ☐ Jumlah tidak bisa melebihi stok

Paraf guru: _____

Exit ticket Pertemuan 3

1. Kenapa `peta.has(objekBaruIsiSama)` bernilai `false`?

---

2. Tulis urutan parameter callback `Map.forEach`: `(_____, _____)`
3. Apa risikonya jika `tambahItem` tidak memeriksa stok?

Pertemuan 4: Rabu 21 Oktober 2026

Tebak dan hitung dengan tangan (SEBELUM menulis kode)

Tabel diskon (4.1):

Subtotal	Persen diskon	Diskon (Rp)
60.000		
74.999		
75.000		
150.000		

Tabel ringkasan (4.2):

Subtotal	Kupon	Diskon	Potongan kupon	Sebelum pembulatan	Pembulatan	Total
100.000	tidak ada					
77.000	tidak ada					
80.000	10% (Level 2)					

Tebak (4.2): `73150 % 100` = _____ . `((a, f = () => 0) => f(a))(5)` = _____

Tebak (4.3): jika `inputBayar.value` berisi `100000`:

Ekspresi	Hasil tebakanmu	Hasil sebenarnya
`inputBayar.value + 1000`		
`Number(inputBayar.value) + 1000`		
`inputBayar.value - 1000`		
`Number("")`		

Jawab (4.1): apa bedanya `3 * 5 / 100` dan `3 * (5 / 100)` di Console? Kenapa urutan pertama lebih aman untuk uang?

Jawab (4.3): apa beda `addEventListener("input", renderKembalian)` dengan `addEventListener("input", renderKembalian())`?

Jawab (4.4): kenapa stok dikurangi setelah pembayaran, bukan saat item masuk nota?

Jawab (4.5, Level 2): kenapa `daftarKupon.get("HEMAT10")(200000)` bisa langsung dipanggil seperti function?

Uji pembayaran (Langkah 4.4)

Skenario	Berhasil?
Bayar saat nota kosong: pesan nota kosong	
Bayar kurang: pesan uang kurang, nota tetap	
Bayar cukup: kembalian benar, nota kosong, stok kartu berkurang	
Beli semua stok Mangga lalu bayar: kartu Mangga menjadi "Habis"	

Checkpoint Pertemuan 4 (diisi guru)

- ☐ `node uji-logika.mjs`: seluruh tes WAJIB ✅ (25 dari 25)
- ☐ Ringkasan nota (subtotal, diskon, pembulatan, total) benar
- ☐ Kembalian dan "uang kurang" benar

Paraf guru: _____

Exit ticket Pertemuan 4

1. `inputBayar.value` bernilai `"100000"`. Hasil `inputBayar.value + 1000` adalah _____ dan `Number(inputBayar.value) + 1000` adalah _____
2. Tulis ternary: diskon 10% jika subtotal >= 150.000, 5% jika >= 75.000, selain itu 0.

3. `73150 % 100` = _____ . Dipakai untuk apa di nota?

Pertemuan 5: Kamis 22 Oktober 2026

Uji silang (Langkah 5.3)

Teman yang kamu uji: _____

Pilih Berhasil (✔) atau Gagal (✘). Jika gagal, tulis apa yang terjadi di kolom catatan. **Jangan memperbaiki karya temanmu.**

No	Skenario	Hasil yang benar	✔/✘	Catatan
1	Buka lewat server, lihat Console	Tidak ada error merah		
2	Lihat katalog	Minimal 6 kartu. Produk stok 0 berlabel "Habis", tombolnya nonaktif		

No	Skenario	Hasil yang benar	✔/✘	Catatan
3	Tambah Apel 2 kali	Nota: Apel 2 kg. Badge Nota menjadi 2		
4	Klik `–` dua kali pada Apel	2 menjadi 1, lalu Apel hilang dari nota		
5	Tambah Mangga (stok 4) 5 kali	Berhenti di 4 kg, ada pesan stok		
6	Apel 3 kg, Anggur 1 kg	Subtotal Rp156.000, diskon 10%, total Rp140.400		
7	Apel 1 kg, Pisang 1 kg, Mangga 1 kg	Subtotal Rp75.000, diskon Rp3.750, pembulatan Rp50, total Rp71.200		
8	Kupon `hemat10` (huruf kecil) lalu kupon `ngawur` (Level 2)	Kupon pertama diterima, kedua menampilkan pesan merah		
9	Bayar kurang, lalu bayar lebih	Pesan "uang kurang", lalu kembalian benar		
10	Setelah bayar	Nota kosong, stok di kartu berkurang		
11	Klik "Mode gelap" dua kali	Gelap, lalu kembali terang. Teks tombol ikut berubah		
12	Lebar layar 375 px (DevTools, `Ctrl + Shift + M`)	Tanpa scroll mendatar. Nota di bawah katalog		

(Angka skenario 6 dan 7 berlaku jika data produk sama dengan tabel di langkah 1.6.)

Bug dari uji silang yang harus kuperbaiki

No skenario	Apa yang terjadi	Sudah diperbaiki?

Persiapan wawancara kode

Pilih tiga fungsi yang kamu tulis sendiri. Isi dengan kata-katamu:

Fungsi (nama dan file)	Menerima	Melakukan	Mengembalikan	Dipakai di
1.				
2.				
3.				

Centang fitur yang sudah kamu selesaikan

Wajib

- ☐ Katalog dari `data.js` (minimal 6 produk)

- [ ] Tambah ke nota dengan `Map`, ubah jumlah, hapus
- [ ] Subtotal, diskon bertingkat, pembulatan, total
- [ ] Bayar dan kembalian (`Number()`)
- [ ] Kode terpecah ke module ES
- [ ] CSS responsif dan mode gelap
- [ ] Sapaan dan papan info

Level 2

- [ ] Kupon (`Map` berisi function, `buatKupon`)

Bonus

- [ ] Filter dan urutan katalog
- [ ] Riwayat pesanan
- [ ] Ganti warna apel

Jelaskan 3 aturan CSS di bagian nota (yang disediakan guru)

Aturan	Menurutku fungsinya
`filter: drop-shadow(...)` pada `.nota`	
`conic-gradient(...)` pada `.nota::after`	
`grid-template-areas` pada `.baris-nota`	

Refleksi (3 sampai 5 kalimat)

1. Bagian proyek yang paling kupahami, dan buktinya (tulis satu potongan kode):

2. Bagian yang masih membingungkan:

3. Rencanaku mempelajarinya:

Exit ticket Pertemuan 5

Jika aku ingin menambah fitur "kupon sekali pakai", file mana yang kuubah dan kenapa?

Checkpoint Pertemuan 5 (diisi guru)

- [ ] Fitur wajib lengkap
- [ ] Uji silang selesai dan bug ditangani
- [ ] Wawancara kode selesai

Paraf guru: _____